BRUCE Humanoid Robot

Sensor Architecture, Dataflow & Process Flow Technical Specification

> **Platform:** BRUCE Open-Platform Bipedal Humanoid | **Simulation Engine:** MuJoCo 3.x | **Framework:** ROS 2 Jazzy | **IPC Transport:** POSIX Shared Memory (`/dev/shm/bruce_sim_state`)

1. System Architecture & End-to-End Pipeline

The simulation utilizes a **two-process decoupled architecture** to ensure that 1000 Hz bipedal contact physics and 60+ FPS interactive third-person game controls run completely isolated from heavy computer vision rendering and LiDAR raycasting loops:

- **Process 1 (MuJoCo Physics & Interactive Game Loop):** Runs physical stepping at 1000 Hz (`dt=0.001s`) and camera orbiting at 60+ FPS. Evaluates balance PD stabilization, walking trajectory state machines, dynamic obstacle dragging, and 72-ray horizontal LiDAR sweeps for local A* obstacle avoidance.
- **Inter-Process Communication (POSIX Shared Memory):** High-speed atomic buffer (`/dev/shm/bruce_sim_state`) carrying a 129-float64 state struct (sim timestamp, base position/quaternion, 16 joint angles/velocities, dynamic objects, and user-selected navigation goals).
- **Process 2 (ROS 2 Perception Bridge):** Dedicated ROS 2 Jazzy node that attaches to the shared memory buffer at 50 Hz. Renders off-screen RGB-D camera images, computes 1,152-ray 3D spherical LiDAR point clouds, broadcasts coordinate frames (TF), and streams standard `sensor_msgs` topics directly into RViz2 and RTAB-Map SLAM.

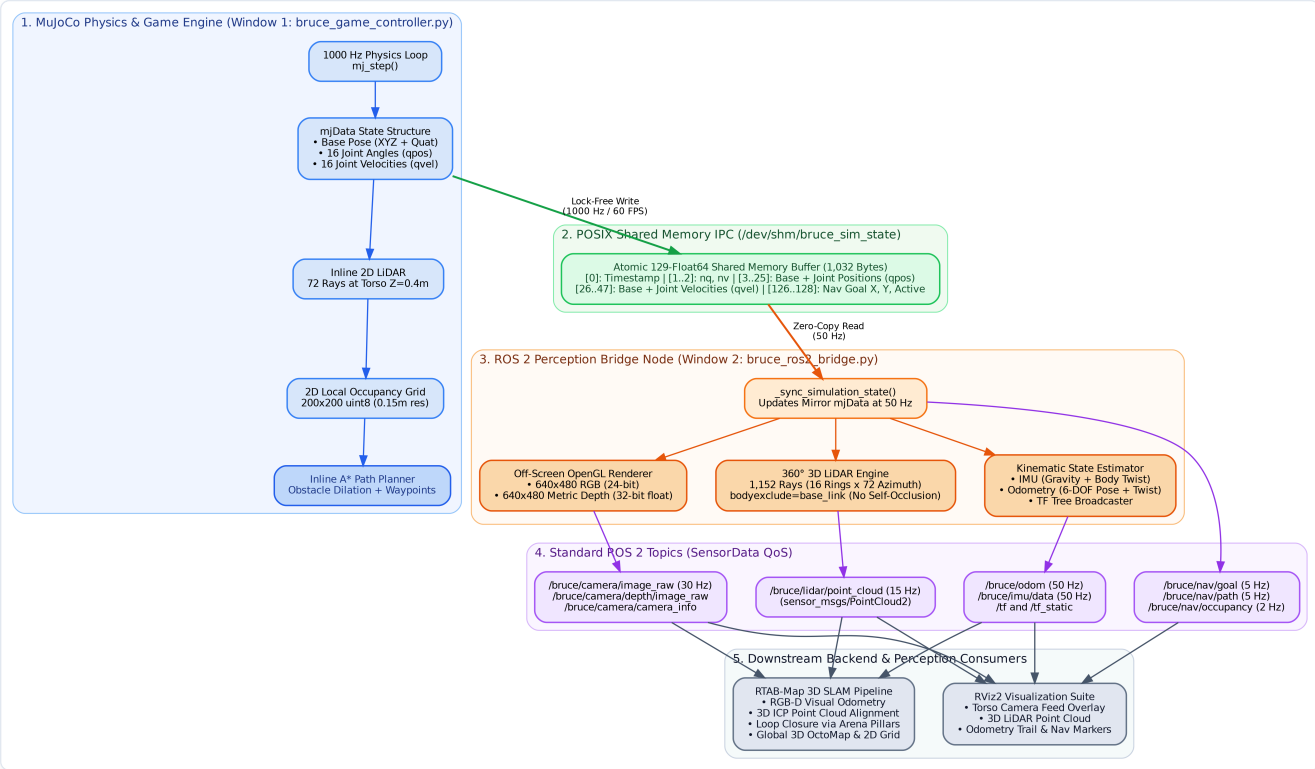


Figure 1: Complete End-to-End Sensor Dataflow and Process Pipeline Across System Boundaries

2. Sensor Specifications & ROS 2 Topic Catalog

The following table provides the technical specifications, physical mounting frames, publishing rates, and message definitions for all active sensor streams:

Sensor Name	Physical Mount	Frame ID	Rate	ROS 2 Topic	ROS 2 Message Type
RGB Head Camera	Torso Upper Base	`head_camera_optical_frame`	30 Hz	`/bruce/camera/image_raw`	`sensor_msgs/msg/Image` (RGB8)
Depth Camera	Torso Upper Base	`head_camera_optical_frame`	30 Hz	`/bruce/camera/depth/image_raw`	`sensor_msgs/msg/Image` (32FC1)
Camera Intrinsics	Calibrated Matrix	`head_camera_optical_frame`	30 Hz	`/bruce/camera/camera_info`	`sensor_msgs/msg/CameraInfo`
360° 3D LiDAR	Torso Crown Mount	`bruce_lidar_link`	15 Hz	`/bruce/lidar/point_cloud`	`sensor_msgs/msg/PointCloud2`
Torso IMU (6-DOF)	Torso Base Origin	`bruce_imu_link`	50 Hz	`/bruce/imu/data`	`sensor_msgs/msg/Imu`
Robot Odometry	Floating Base EKF	`odom` → `base_link`	50 Hz	`/bruce/odom`	`nav_msgs/msg/Odometry`
Coordinate Frames	Dynamic & Static	Multi-frame	50 Hz	`/tf`, `/tf_static`	`tf2_msgs/msg/TFMessage`
Local 2D Nav Grid	Inline Raycaster	`odom`	2 Hz	`/bruce/nav/occupancy`	`nav_msgs/msg/OccupancyGrid`
Nav Goal Target	Interactive Picker	`odom`	5 Hz	`/bruce/nav/goal`	`geometry_msgs/msg/PoseStamped`

3. Detailed Sensor Processing Pipelines

3.1 RGB-D Head Camera Pipeline

The vision system is defined in `bruce.xml` on the torso base link: `<camera name="head_camera" pos="0.08 0 0.16" fovy="65" xyaxes="0 -1 0 0 0 1"/>`. The optical frame uses standard robotics orientation (+Z pointing forward, +X rightward, +Y downward).

- **Off-Screen Context Synchronization:** The bridge synchronizes mirror kinematics and invokes `renderer.update_scene(self.data, camera="head_camera")`.
- **Color Video (RGB8):** `renderer.render()` captures a 640×480×3 uint8 frame (1,920-byte row stride), published at 30 Hz.
- **Metric Depth (32FC1):** `renderer.enable_depth_rendering()` extracts true Euclidean distance in meters per pixel as 32-bit floats (2,560-byte row stride), published at 30 Hz.
- **Intrinsics Matrix:** Focal lengths (`fx = fy = 476.7 px`) and principal center (`cx = 320, cy = 240`) are published via `CameraInfo`.

3.2 360° Multi-Beam 3D LiDAR Pipeline

Mounted on the torso crown at `bruce_lidar_link` (`pos=[0.02, 0, 0.22]`). Generates 1,152 rays per sweep (16 elevation rings × 72 azimuth steps):

- **Pre-computed Geometry:** 16 vertical rings (-16° to +16° in ~2.13° steps) × 72 horizontal steps (5° steps).
- **Raycasting & Self-Occlusion Avoidance:** Each local ray is rotated into world coordinates by the site orientation matrix and evaluated using `mujoco.mj_ray(..., bodyexclude=self.base_body_id)`. Passing `bodyexclude` ensures rays do not self-intersect the robot's own head, visor, or chest.
- **PointCloud2 Serialization:** Hits between 0.15m and 25.0m are converted to IEEE 754 float32 triples and published on `/bruce/lidar/point_cloud` at 15 Hz.

3.3 Coordinate Tree & Spatial Transforms (TF)

Dynamic transforms represent the robot moving through the world, while static transforms represent rigid sensor mountings:

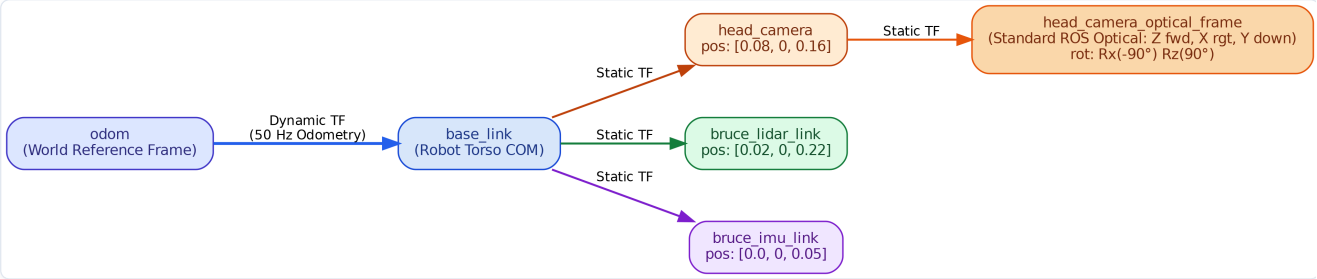


Figure 2: TF Coordinate Tree and Spatial Transform Relationships

4. POSIX Shared Memory Buffer Specification

The memory segment (`/dev/shm/bruce_sim_state`) provides lock-free, zero-copy IPC between Process 1 and Process 2:

Index Range	Type	Count	Field Description	Update Rate
`[0]`	float64	1	Simulation Timestamp (seconds)	1000 Hz
`[1 .. 2]`	float64	2	Model Dimensions (nq=23, nv=22)	Static
`[3 .. 5]`	float64	3	Robot Torso Base Position (X, Y, Z in meters)	1000 Hz
`[6 .. 9]`	float64	4	Robot Torso Orientation Quaternion (w, x, y, z)	1000 Hz
`[10 .. 25]`	float64	16	16 Actuated Joint Angles (qpos in radians)	1000 Hz
`[26 .. 28]`	float64	3	Torso Base Linear Velocity (vx, vy, vz in m/s)	1000 Hz
`[29 .. 31]`	float64	3	Torso Base Angular Velocity (wx, wy, wz in rad/s)	1000 Hz
`[32 .. 47]`	float64	16	16 Actuated Joint Velocities (qvel in rad/s)	1000 Hz
`[48 .. 125]`	float64	78	Arena Dynamic Rigid Bodies (Ball, Crates, Barrels)	1000 Hz
`[126]`	float64	1	Navigation Goal X Coordinate (meters)	On Change
`[127]`	float64	1	Navigation Goal Y Coordinate (meters)	On Change
`[128]`	float64	1	Navigation Goal Active Flag (1.0 = Active, 0.0 = Off)	On Change

5. Downstream Backend Consumers

- **RViz2 3D Dashboard:** Visualizes live RGB video feeds, colored 3D LiDAR point clouds, odometry trajectory trails, the TF coordinate tree, local 2D occupancy costmaps, and active 3D navigation target beacons.
- **RTAB-Map 3D SLAM Suite:** Couples RGB-D visual feature tracking with 3D LiDAR point cloud registration (ICP). Detects loop closures using the arena's 4 visual landmark columns to eliminate odometry drift and generate persistent 3D OctoMaps and 2D grid maps for multi-room navigation.